

Guía de Laboratorio: Seguridad en Autenticación Web

Tema: Autenticación Segura, Hashing de Contraseñas y Protección contra Fuerza Bruta

Objetivo General

Diseñar e implementar un sistema de autenticación seguro para una aplicación web, aplicando técnicas modernas de hashing de contraseñas mediante algoritmos con salt automático (BCrypt) y desarrollando mecanismos de defensa perimetral en el servidor mediante alguna método escogido previamente, para mitigar ataques de fuerza bruta.

Introducción Teórica y Nota Arquitectónica Importante

Antes de iniciar con los componentes del laboratorio, es fundamental establecer una premisa clave de la arquitectura de seguridad web:

Nota Arquitectónica Obligatoria: Toda la lógica de autenticación, verificación de credenciales, hashing de contraseñas y control de intentos fallidos **debe procesarse exclusivamente en el lado del servidor** (backend) utilizando lenguajes y frameworks como Node.js (Express), PHP (Laravel), Python (Django/FastAPI) o Java (Spring Boot). Los lenguajes del lado del cliente como HTML, CSS y JavaScript ejecutado en el navegador no son seguros para este propósito, ya que el código fuente puede ser inspeccionado, modificado o ignorado completamente por un atacante a través de las herramientas de desarrollo del navegador o mediante solicitudes HTTP directas.

El proceso de autenticación segura involucra los siguientes principios teóricos fundamentalmente distintivos:

- **Hashing Unidireccional:** A diferencia del cifrado simétrico (como Blowfish o AES) que permite recuperar el texto claro mediante una clave, una función hash criptográfica es una función matemática de una sola vía. Una vez generada la cadena hash, es computacionalmente inviable revertirla para obtener la contraseña original.
- **Salt:** Consiste en concatenar una secuencia aleatoria y única de caracteres a la contraseña de cada usuario antes de aplicarle el algoritmo de hash. Esto garantiza que dos usuarios con la misma contraseña generen hashes totalmente diferentes, anulando el uso de tablas arcoíris (Rainbow Tables) y ataques de diccionario masivos precomputados.

Sección 1: Configuración de la Base de Datos y Diseño del Formulario de Login

1.1. Esquema de la Base de Datos

Para gestionar la autenticación y el control de acceso basado en roles (RBAC), es necesario estructurar la base de datos relacional con al menos dos tablas principales: roles y usuarios.

Tabla	Campo	Tipo de Dato	Restricciones	Descripción
roles	id	INT / SERIAL	PRIMARY KEY, AUTO_INCREMENT	Identificador único del rol
roles	nombre	VARCHAR(50)	NOT NULL, UNIQUE	Nombre del rol (Ej: 'admin', 'usuario')
usuarios	id	INT / SERIAL	PRIMARY KEY, AUTO_INCREMENT	Identificador único del usuario
usuarios	username	VARCHAR(50)	NOT NULL, UNIQUE	Nombre de usuario o correo electrónico
usuarios	password_hash	VARCHAR(255)	NOT NULL	Cadena completa del hash generado por BCrypt
usuarios	rol_id	INT	FOREIGN KEY -> roles(id)	Rol asignado al usuario

Tabla	Campo	Tipo de Dato	Restricciones	Descripción
usuarios	intentos_fallidos	INT	DEFAULT 0	Contador de fallos de inicio de sesión
usuarios	bloqueado_hasta	DATETIME / TIMESTAMP	NULLABLE	Fecha y hora de finalización del bloqueo temporal

Script SQL sugerido para la creación de la estructura:

```
CREATE TABLE roles (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(50) NOT NULL UNIQUE
);
```

```
CREATE TABLE usuarios (
  id INT AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(50) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  rol_id INT NOT NULL,
  intentos_fallidos INT DEFAULT 0,
  bloqueado_hasta DATETIME NULL,
  FOREIGN KEY (rol_id) REFERENCES roles(id)
);
```

```
INSERT INTO roles (nombre) VALUES ('Administrador'), ('Usuario');
```

1.2. Diseño del Formulario de Inicio de Sesión (HTML5)

El formulario del lado del cliente captura las credenciales y las envía mediante el método POST hacia el endpoint seguro del servidor. Se deben deshabilitar características del navegador que puedan comprometer la privacidad del usuario.

```
<!DOCTYPE html>
<html lang="es">
<head>
```

```
<meta charset="UTF-8">
<title>Inicio de Sesión Seguro</title>
</head>
<body>
    <h2>Acceso al Sistema</h2>
    <form action="/login" method="POST">
        <div>
            <label for="username">Usuario / Email:</label>
            <input type="text" id="username" name="username"
required autocomplete="username">
        </div>
        <br>
        <div>
            <label for="password">Contraseña:</label>
            <input type="password" id="password" name="password"
required autocomplete="current-password">
        </div>
        <br>
        <button type="submit">Iniciar Sesión</button>
    </form>
</body>
</html>
```

Sección 2: Implementación de Hashing de Contraseñas con BCrypt

2.1. Anatomía del Algoritmo BCrypt y Manejo Integrado del Salt

BCrypt se basa en el cifrado de bloques EksBlowfishSetup (Expensive Key Schedule Blowfish). Una de las ventajas principales de BCrypt es que embebe automáticamente el costo computacional (factor de trabajo) y el *salt* dentro del texto final generado. Una cadena hash típica de BCrypt luce de la siguiente manera:

```
$2b$10$N9qo8uLOickgx2ZMRZoMyeIjZAgcfl7p92ldGxad68LJZdL17lhWy
\__/\  \/\  \_____/ \_____/
Alg Costo      Salt (22 chars)      Hash (31 chars)
```

- \$2b\$: Identificador de la revisión del algoritmo BCrypt.
- \$10\$: Factor de costo ($2^{10} = 1,024$ rondas de iteración).
- N9qo8uLOickgx2ZMRZoMye: Salt único de 128 bits codificado en Base64.

- `ljZAgcfI7p92ldGxad68LJZdL17lhWy`: Hash resultante de la contraseña combinada con el salt. Al momento de autenticar un usuario, no se requiere almacenar el salt en una columna separada. La librería del servidor lee la cadena almacenada en `password_hash`, extrae automáticamente el salt y el factor de costo, procesa la contraseña ingresada por el usuario con esos mismos parámetros y compara si ambos hashes coinciden de forma transparente.

2.2. Ejemplo de Implementación en el Servidor (Node.js con Express y bcrypt)

```
const bcrypt = require('bcrypt');
const SALT_ROUNDS = 10; // Factor de trabajo recomendable

// 1. Proceso de Registro (Generación del Hash)
async function registrarUsuario(passwordPlano) {
  // Genera el salt y el hash en un solo paso
  const hash = await bcrypt.hash(passwordPlano, SALT_ROUNDS);
  return hash; // Este valor completo se guarda en la columna
password_hash
}

// 2. Proceso de Verificación durante el Login
async function verificarPassword(passwordIngresado,
hashAlmacenado) {
  // bcrypt extrae el salt del hashAlmacenado y comprueba la
coincidencia
  const esValido = await bcrypt.compare(passwordIngresado,
hashAlmacenado);
  return esValido; // Retorna true o false
}
```

2.3. Ejemplo Equivalente en PHP

```
<?php
// Hashing al crear o actualizar la contraseña
$passwordPlano = $_POST['password'];
$hashAlmacenado = password_hash($passwordPlano, PASSWORD_BCRYPT,
['cost' => 10]);

// Verificación durante el login
$passwordIngresado = $_POST['password'];
if (password_verify($passwordIngresado, $hashAlmacenado)) {
```

```
// Contraseña correcta
} else {
  // Contraseña incorrecta
}
?>
```

Sección 3: Protección contra Ataques de Fuerza Bruta mediante Middleware

Un ataque de fuerza bruta intenta adivinar contraseñas mediante pruebas automatizadas repetitivas. Para mitigar esta amenaza, el servidor debe interceptar las solicitudes entrantes utilizando un **modulo intermedio o middleware**

3.1. Concepto y Funcionamiento del Middleware

Un Middleware es una capa de código intermedio que se ejecuta antes de que una solicitud HTTP alcance el controlador principal de la ruta de login. Evalúa metadatos como la dirección IP origen o la tasa de peticiones y decide si permite continuar el procesamiento o bloquea la conexión inmediatamente.

3.2. Estrategias de Bloqueo Implementadas

1. **Limitación de Tasa (Rate Limiting) por IP:** Registrar el número de peticiones enviadas desde una dirección IP hacia la ruta /login en una ventana de tiempo determinada (ejemplo: máximo 5 intentos cada 15 minutos).
2. **Bloqueo Temporal y Retardo (Cooldown):** Si se supera el número máximo de fallos permitidos, la dirección IP se registra en una lista de bloqueo temporal (o en una memoria caché como Redis / Map en memoria), devolviendo un código de estado HTTP 429 Too Many Requests.

3.3. Ejemplo de Middleware en Node.js / Express

```
const intentosIP = new Map(); // Almacenamiento temporal en memoria

function middlewareProteccionFuerzaBruta(req, res, next) {
  const ip = req.ip || req.connection.remoteAddress;
  const ahora = Date.now();
  const VENTANA_TIEMPO = 15 * 60 * 1000; // 15 minutos en milisegundos
```

```
const MAX_INTENTOS = 5;

if (!intentosIP.has(ip)) {
  intentosIP.set(ip, { conteo: 0, primerIntento: ahora,
bloqueadoHasta: 0 });
}

const registro = intentosIP.get(ip);

// Verificar si la IP está bloqueada actualmente
if (registro.bloqueadoHasta > ahora) {
  const segundosRestantes =
Math.ceil((registro.bloqueadoHasta - ahora) / 1000);
  return res.status(429).json({
    error: `Demasiados intentos fallidos. Su IP ha sido
bloqueada temporalmente. Intente de nuevo en
${segundosRestantes} segundos.`
  });
}

// Reiniciar conteo si expiró la ventana de tiempo
if (ahora - registro.primerIntento > VENTANA_TIEMPO) {
  registro.conteo = 0;
  registro.primerIntento = ahora;
}

req.registroIP = registro;
next();
}

module.exports = middlewareProteccionFuerzaBruta;
```

Resumen de Entregables para la Práctica de Laboratorio

Con la página web de cifrado clásico, se debe agregar el modulo de bloqueo d ataque de fuerza bruta, que cumpla con los requerimientos de diseño, donde al menos tenga una manera de detener el ataque de fuerza bruta.

- Script SQL completo con la creación de tablas e inserción de datos iniciales.
- Código fuente del backend (Node.js o PHP) implementando el hashing de contraseñas con

BCrypt.

- Middleware funcional de protección contra fuerza bruta con prueba demostrativa de bloqueo de IP tras 5 intentos fallidos consecutivos.
- Documento de conclusiones justificando por qué las validaciones de seguridad no deben realizarse en el navegador (cliente HTML/JS).